

Plantilla de informe EP1 - TechMarket SpA

Completar por el equipo sin modificar la estructura principal

Portada

- Asignatura: BDY1103 Taller de Base de Datos
- Evaluación Parcial N°1
- Equipo / integrantes / sección / docente / fecha

Respuesta del estudiante:

Asignatura	BDY1103 Taller de Base de Datos
Evaluación	Evaluación Parcial N°1
Integrantes	Danyer Córdoba Max Herrera Pablo Quintino Tomás Salas
Sección	008D
Docente	Leonel Octavio Garay Burgos
Fecha	23/09/2026

1. Contexto del problema

- Explique qué necesidad tiene TechMarket.
- Indique qué datos se procesarán y qué información se espera generar.

Respuesta del estudiante:

TechMarket SpA es una empresa de retail tecnológico con tres sucursales: Antonio Varas (Providencia), Plaza Oeste (Cerrillos) y Viña Centro (Viña del Mar). La empresa necesita procesar automáticamente las ventas de cada sucursal en un período determinado para identificar a sus clientes según su nivel de compra y asignarles un beneficio comercial, dejando registro de cada asignación y de quién la realizó.

Hacer este proceso a mano o con consultas sueltas es lento y propenso a errores: un mismo cliente puede comprar varias veces en el período y en distintas sucursales, y el porcentaje de beneficio depende del monto acumulado. Por eso la solución se construye en la base de datos con PL/SQL, de modo que las reglas de negocio queden centralizadas y el resultado quede guardado y auditado.

Datos que se procesan

- **SUCURSAL:** permite filtrar el proceso por sede.
- **CLIENTE:** identifica a quién se le consolida la compra.
- **VENTA:** cabecera de cada transacción (fecha, sucursal, cliente y total); es la base del cálculo por período.
- **DETALLE_VENTA y PRODUCTO:** productos vendidos en cada transacción (relación maestro-detalle).

Información que se genera

Por cada cliente con ventas en la sucursal y el período consultados se obtiene la cantidad de compras, el total acumulado, la categoría comercial y el porcentaje de beneficio. Ese resultado se guarda en BITACORA_BENEFICIOS, y cada inserción o actualización en la bitácora queda registrada automáticamente en AUDITORIA_BENEFICIOS (trazabilidad).

Categoría	Rango de compras acumuladas	Beneficio
BRONZE	Menos de \$300.000	5%
SILVER	\$300.000 a \$699.999	10%
GOLD	\$700.000 a \$1.499.999	15%
PLATINUM	\$1.500.000 o más	20%

Tabla 1. Reglas de negocio para la clasificación de clientes.

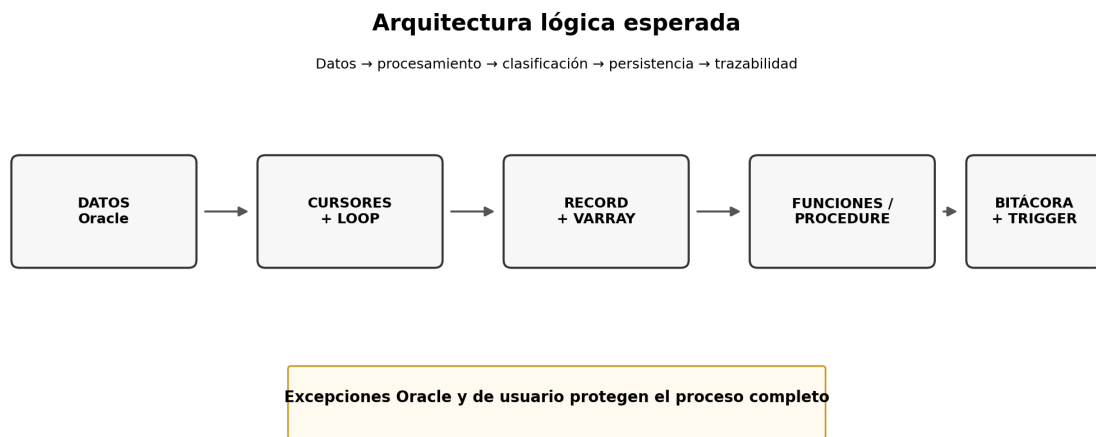


Figura 1. Arquitectura lógica: datos → procesamiento → clasificación → persistencia → trazabilidad.

2. Modelo de datos

- Inserte el diagrama entregado.
- Explique tablas principales, PK, FK y relaciones usadas.

Respuesta del estudiante:

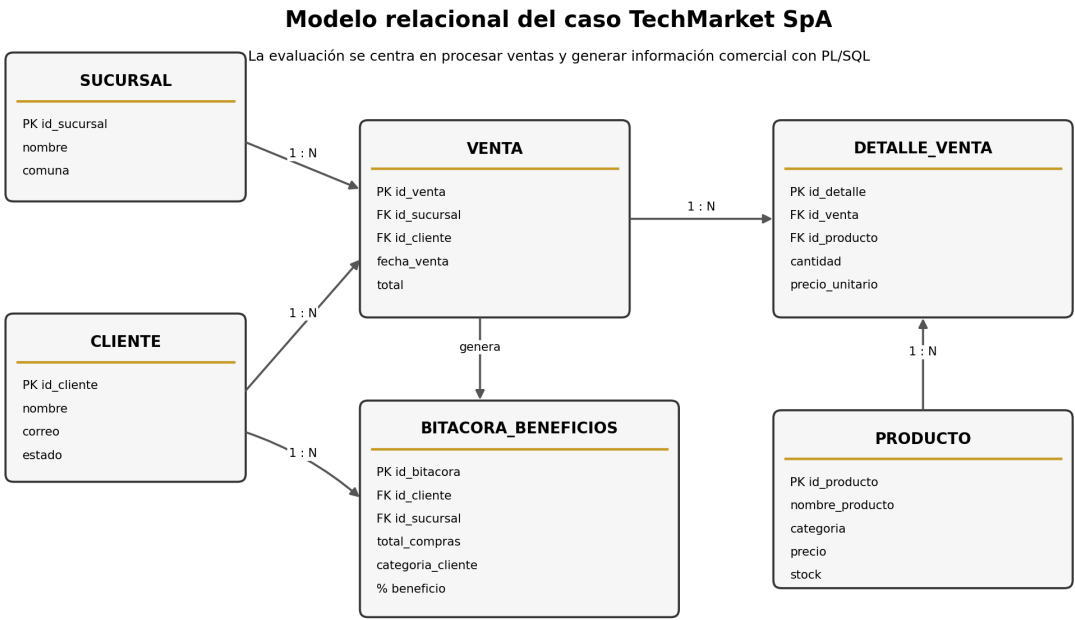


Figura 2. Modelo relacional del caso TechMarket SpA (entregado por el docente).

Tabla	PK	FK	Rol en el negocio
SUCURSAL	id_sucursal	—	Punto de venta. Filtra el proceso por sede.
CLIENTE	id_cliente	—	Persona que compra. Estado ACTIVO/INACTIVO (CHECK) y correo único (UNIQUE).
PRODUCTO	id_producto	—	Catálogo tecnológico. CHECK precio > 0 y stock ≥ 0.
VENTA	id_venta	id_sucursal → SUCURSAL; id_cliente → CLIENTE	Cabecera de la transacción; base del cálculo por periodo.
DETALLE_VENTA	id_detalle	id_venta → VENTA; id_producto → PRODUCTO	Productos vendidos por transacción (maestro-detalle).
BITACORA_BENEFICIOS	id_bitacora (IDENTITY)	id_cliente → CLIENTE; id_sucursal → SUCURSAL	Resultado del proceso: compras, total, categoría y % de beneficio.
AUDITORIA_BENEFICIOS	id_auditoria (IDENTITY)	— (id_bitacora como referencia lógica)	Trazabilidad: operación, usuario, fecha y valores anterior/nuevo.

Tabla 2. Tablas, claves y rol de cada una.

Relaciones utilizadas

- SUCURSAL 1:N VENTA y CLIENTE 1:N VENTA:** una sucursal registra muchas ventas y un cliente puede comprar muchas veces. Son las relaciones que recorren los cursores para consolidar compras por cliente, sucursal y periodo.
- VENTA 1:N DETALLE_VENTA y PRODUCTO 1:N DETALLE_VENTA:** estructura maestro-detalle de cada transacción.
- CLIENTE 1:N BITACORA_BENEFICIOS y SUCURSAL 1:N BITACORA_BENEFICIOS:** cada ejecución del proceso genera un registro por cliente y sucursal procesados.

- **BITACORA_BENEFICIOS → AUDITORIA_BENEFICIOS:** el trigger inserta un registro de auditoría por cada cambio en la bitácora. Esta tabla viene en el script base, pero no aparece en el diagrama; no tiene FK declarada, así que la auditoría se conserva aunque se elimine el registro de la bitácora.

El script base también crea índices sobre VENTA (id_sucursal, fecha_venta) y VENTA (id_cliente), que corresponden exactamente a los filtros que usan los cursores, y un índice sobre DETALLE_VENTA (id_venta).

3. RECORD y VARRAY

- Pegue el fragmento de código desarrollado.
- Explique qué representa el RECORD.
- Explique por qué utilizó VARRAY y qué límite posee.
- Inserte evidencia de ejecución.

Respuesta del estudiante:

Fragmento del bloque anónimo desarrollado (sección 9 del script maestro, responsable: Max Herrera). Se muestran la declaración y el procesamiento; las líneas de DBMS_OUTPUT están en el Anexo A.

DECLARE

```
TYPE cliente_record IS RECORD (  
    id_cliente CLIENTE.ID_CLIENTE%TYPE,  
    nombre_cliente CLIENTE.NOMBRE%TYPE,  
    cantidad_compras NUMBER,  
    total_comprado NUMBER,  
    categoria VARCHAR2(20),  
    beneficio NUMBER  
);  
  
v_cliente cliente_record;  
  
TYPE beneficios_varray IS VARRAY(4) OF NUMBER;  
  
v_beneficios beneficios_varray :=  
    beneficios_varray(5, 10, 15, 20);
```

BEGIN

```
SELECT  
    c.id_cliente,  
    c.nombre,  
    COUNT(v.id_venta),  
    NVL(SUM(v.total), 0)  
  
INTO  
    v_cliente.id_cliente,  
    v_cliente.nombre_cliente,  
    v_cliente.cantidad_compras,  
    v_cliente.total_comprado  
  
FROM cliente c
```

¿Qué representa el RECORD?

El tipo `cliente_record` representa la ficha comercial de un cliente: agrupa en una sola variable su identificador, nombre, cantidad de compras, total comprado, categoría y porcentaje de beneficio. Los campos `id_cliente` y `nombre_cliente` usan `%TYPE`, así heredan el tipo de la columna de `CLIENTE` y el bloque sigue funcionando si ese tipo cambia.

El `RECORD` se usa realmente: la consulta con `LEFT JOIN` y `GROUP BY` carga sus cuatro primeros campos con `SELECT ... INTO`, y luego las funciones `calcular_categoria` y `calcular_beneficio` completan la categoría y el beneficio. En vez de manejar seis variables sueltas, el cliente se trata como una unidad, que es la forma natural de representar un registro del negocio.

¿Por qué VARRAY y qué límite tiene?

El tipo `beneficios_varray` se declaró como `VARRAY(4) OF NUMBER` y se inicializa con (5, 10, 15, 20), los porcentajes de las cuatro categorías. Se eligió un `VARRAY` porque el catálogo de beneficios es una lista pequeña, ordenada y de tamaño conocido: justamente el caso para el que sirve esta colección.

Límite: como máximo 4 elementos, fijado en la declaración. Intentar agregar un quinto con `EXTEND` produce el error `ORA-06532` (subíndice fuera del límite). Los elementos se acceden por posición, del 1 al 4, y el orden se conserva.

Limitación detectada: en la versión actual el `VARRAY` sirve como catálogo de referencia y se muestra en la salida, pero el porcentaje del cliente lo calcula la función `calcular_beneficio`. Como mejora (sección 7), el beneficio podría obtenerse directamente del `VARRAY` según la posición de la categoría (por ejemplo `v_beneficios(3)` para `GOLD`), dejando el catálogo como fuente única de los porcentajes.

Evidencia de ejecución

Output:

```
ID Cliente: 101
Nombre: Ana Torres
Cantidad compras: 4
Total comprado: $1449940
Categoria: GOLD
Beneficio: 15%
Beneficios disponibles: 5%, 10%, 15%, 20%
```

Evidencia 1. Salida del bloque `RECORD + VARRAY` (cliente 101), ejecutado en `OneCompiler` (Oracle).

Nota: el bloque consolida todas las ventas del cliente 101 sin filtrar por sucursal ni período (3 ventas en la sucursal 10 y 1 en la sucursal 20), por eso el total es \$1.449.940 y no el de agosto en la sucursal 10.

4. Cursores explícitos complejos

- Indique parámetros del cursor.
- Explique qué recorre el `LOOP` externo.

- Explique qué recorre el LOOP interno.
- Inserte evidencia del resultado.

Respuesta del estudiante:

Se implementaron dos cursores explícitos parametrizados que trabajan como maestro-detalle (sección 10 del script maestro, responsable: Max Herrera).

```
DECLARE
```

```
    v_id_sucursal NUMBER := 10;
```

```
    v_fecha_desde DATE :=  
        DATE '2026-08-01';
```

```
    v_fecha_hasta DATE :=  
        DATE '2026-08-31';
```

```
-- CURSOR MAESTRO
```

```
CURSOR cur_clientes(  
    p_id_sucursal NUMBER,  
    p_fecha_desde DATE,  
    p_fecha_hasta DATE  
)
```

```
IS
```

```
    SELECT DISTINCT  
        c.id_cliente,  
        c.nombre
```

```
    FROM cliente c
```

```
    JOIN venta v
```

```
        ON c.id_cliente = v.id_cliente
```

```
    WHERE v.id_sucursal = p_id_sucursal
```

```
    AND v.fecha_venta  
        BETWEEN p_fecha_desde  
        AND p_fecha_hasta
```

```
    ORDER BY c.id_cliente;
```

Parámetros de los cursores

Cursor	Parámetros	Función
cur_clientes (maestro)	p_id_sucursal, p_fecha_desde, p_fecha_hasta	Clientes distintos que compraron en la sucursal dentro del período.
cur_ventas (detalle)	p_id_cliente, p_id_sucursal, p_fecha_desde, p_fecha_hasta	Ventas de un cliente específico en esa sucursal y período.

Tabla 3. Cursores y parámetros. Valores de prueba: sucursal 10, 01-08-2026 al 31-08-2026.

LOOP externo

Recorre cur_clientes: un cliente por iteración. Por cada cliente reinicia los acumuladores (v_cantidad_compras y v_total_comprado), muestra su identificación y, al terminar el LOOP interno, informa el total, la categoría y el beneficio usando las funciones calcular_categoria y calcular_beneficio.

LOOP interno

Recorre cur_ventas, que recibe el id_cliente de la iteración actual del LOOP externo. Por cada venta de ese cliente en la sucursal y el período, muestra el número, la fecha y el monto, y acumula la cantidad y el total. Así el detalle depende del registro maestro, que es lo que caracteriza a un cursor anidado.

```
BEGIN

-- LOOP MAESTRO

FOR cliente IN cur_clientes(

    v_id_sucursal,

    v_fecha_desde,

    v_fecha_hasta

)

LOOP

    v_cantidad_compras := 0;

    v_total_comprado := 0;

    DBMS_OUTPUT.PUT_LINE(

        '-----'

    );

    DBMS_OUTPUT.PUT_LINE(

        'Cliente: ' ||

        cliente.id_cliente ||

        ' - ' ||

        cliente.nombre

    );

-- LOOP DETALLE

FOR venta IN cur_ventas(

    cliente.id_cliente,

    v_id_sucursal,

    v_fecha_desde,

    v_fecha_hasta

)

LOOP
```

Los parámetros modifican el resultado

Con los mismos cursores, cambiar la sucursal o el período produce otro conjunto de clientes y totales:

Cliente	Sucursal 10 – agosto 2026	Sucursal 20 – agosto 2026
101 Ana Torres	3 compras · \$1.339.950 · GOLD 15%	1 compra · \$109.990 · BRONZE 5%
102 Pedro Diaz	2 compras · \$759.960 · GOLD 15%	—
103 Maria Soto	1 compra · \$219.980 · BRONZE 5%	1 compra · \$319.970 · SILVER 10%
104 Carlos Rojas	1 compra · \$1.599.800 · PLATINUM 20%	—

Tabla 4. Resultado esperado según los datos iniciales del caso.

Evidencia del resultado

```

-----
Cliente: 101 – Ana Torres
Venta: 5001 | Fecha: 03/08/2026 | Total: $899990
Venta: 5002 | Fecha: 12/08/2026 | Total: $279980
Venta: 5003 | Fecha: 25/08/2026 | Total: $159980
Cantidad compras: 3
Total comprado: $1339950
Categoria: GOLD
Beneficio: 15%
-----

```

Evidencia 2a. Cursores, sucursal 10, agosto 2026: cliente 101.

```

-----
Cliente: 102 – Pedro Diaz
Venta: 5004 | Fecha: 07/08/2026 | Total: $239990
Venta: 5005 | Fecha: 19/08/2026 | Total: $519970
Cantidad compras: 2
Total comprado: $759960
Categoria: GOLD
Beneficio: 15%
-----

```

```

-----
Cliente: 103 – Maria Soto
Venta: 5006 | Fecha: 11/08/2026 | Total: $219980
Cantidad compras: 1
Total comprado: $219980
Categoria: BRONZE
Beneficio: 5%
-----

```

```

-----
Cliente: 104 – Carlos Rojas
Venta: 5007 | Fecha: 15/08/2026 | Total: $1599800
Cantidad compras: 1
Total comprado: $1599800
Categoria: PLATINUM
-----

```

Evidencia 2b. Cursores, sucursal 10, agosto 2026: clientes 102, 103 y 104.

5. Excepciones

- Identifique una excepción Oracle utilizada.
- Identifique una excepción definida por usuario.
- Explique caso normal y caso de error probado.
- Inserte evidencias.

Respuesta del estudiante:

El control de errores se probó con cuatro casos (P1 a P4, secciones 15 a 18 del script maestro, responsable: Danyer Córdoba). Cada bloque atrapa su error con un mensaje comprensible y además tiene WHEN OTHERS como respaldo para cualquier error no previsto (muestra SQLERRM).

Prueba	Tipo	Excepción	Situación
P1	Caso normal	—	Procesamiento de la sucursal 10, agosto 2026, con el package.
P2	Error	NO_DATA_FOUND (Oracle, predefinida)	Se consulta la sucursal 999, que no existe.
P3	Error	ex_periodo_invalido (usuario)	Fecha de inicio (31-08) posterior a la de fin (01-08).
P4	Error	ex_sin_ventas (usuario)	Sucursal 20 sin ventas en septiembre de 2026.

Tabla 5. Casos de prueba de excepciones.

Excepción Oracle: NO_DATA_FOUND (P2)

Oracle la lanza automáticamente cuando un `SELECT ... INTO` no devuelve filas. Se usa para validar que la sucursal exista antes de procesar.

```
DECLARE

    v_sucursal_id NUMBER := 999;

    v_sucursal NUMBER;

BEGIN

    SELECT id_sucursal
    INTO v_sucursal

    FROM sucursal

    WHERE id_sucursal = v_sucursal_id;

    DBMS_OUTPUT.PUT_LINE(
        'Sucursal encontrada: ' ||
        v_sucursal
    );

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        DBMS_OUTPUT.PUT_LINE(
            'ERROR: La sucursal ' ||
            v_sucursal_id ||
            ' no existe en el sistema.'
        );

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE(
            'ERROR NO ESPERADO: ' ||
```

ERROR: La sucursal 999 no existe en el sistema.

Evidencia 3. P2 – NO_DATA_FOUND con la sucursal 999.

Excepciones definidas por el usuario (P3 y P4)

Se declaran con **EXCEPTION** en la sección **DECLARE** y se disparan con **RAISE** cuando se incumple una regla de negocio que Oracle no detecta por sí solo: **ex_periodo_invalido** valida que el rango de fechas sea coherente y **ex_sin_ventas** evita procesar una sucursal sin movimientos en el período.

```
DECLARE
```

```
    v_fecha_inicio DATE :=  
        DATE '2026-08-31';
```

```
    v_fecha_fin DATE :=  
        DATE '2026-08-01';
```

```
    ex_periodo_invalido EXCEPTION;
```

```
BEGIN
```

```
    IF v_fecha_inicio > v_fecha_fin THEN
```

```
        RAISE ex_periodo_invalido;
```

```
    END IF;
```

```
    DBMS_OUTPUT.PUT_LINE(  
        'Periodo valido.'  
    );
```

```
EXCEPTION
```

```
    WHEN ex_periodo_invalido THEN
```

```
        DBMS_OUTPUT.PUT_LINE(  
            'ERROR: El periodo es invalido.'  
        );
```

```
        DBMS_OUTPUT.PUT_LINE(  
            'La fecha de inicio no puede ser posterior a la fecha de fin.'  
        );
```

ERROR: El periodo es invalido.
La fecha de inicio no puede ser posterior a la fecha de fin.

Evidencia 4. P3 – ex_periodo_invalido.

```
DECLARE
```

```
    v_sucursal_id NUMBER := 20;
```

```
    v_fecha_inicio DATE :=
```

```
        DATE '2026-09-01';
```

```
    v_fecha_fin DATE :=
```

```
        DATE '2026-09-30';
```

```
    v_total_ventas NUMBER;
```

```
    ex_sin_ventas EXCEPTION;
```

```
BEGIN
```

```
    SELECT COUNT(*)
```

```
    INTO v_total_ventas
```

```
    FROM venta
```

```
    WHERE id_sucursal = v_sucursal_id
```

```
    AND fecha_venta
```

```
        BETWEEN v_fecha_inicio
```

```
        AND v_fecha_fin;
```

```
    IF v_total_ventas = 0 THEN
```

```
        RAISE ex_sin_ventas;
```

```
    END IF;
```

ERROR: La sucursal 20 no registro ventas en el periodo.

Evidencia 5. P4 – ex_sin_ventas con la sucursal 20 en septiembre.

Caso normal (P1)

Con parámetros válidos (sucursal 10, 01-08-2026 al 31-08-2026) no se dispara ninguna excepción: el package procesa a los cuatro clientes, inserta sus registros en la bitácora, el trigger registra cada inserción en la auditoría y la transacción se confirma con COMMIT.

```
BEGIN

    techmarket_beneficios.procesar_beneficios(

        10,

        DATE '2026-08-01',

        DATE '2026-08-31'

    );

END;

/

COMMIT;
```

TRAZABILIDAD: Registro automatizado en AUDITORIA_BENEFICIOS para ID 1
TRAZABILIDAD: Registro automatizado en AUDITORIA_BENEFICIOS para ID 2
TRAZABILIDAD: Registro automatizado en AUDITORIA_BENEFICIOS para ID 3
TRAZABILIDAD: Registro automatizado en AUDITORIA_BENEFICIOS para ID 4

Evidencia 6. P1 – Caso normal: el trigger registra en la auditoría cada inserción del package en la bitácora.

Resguardo de la consistencia de datos

- Los casos de error P2 a P4 se detectan antes de cualquier escritura, así que no dejan registros incompletos en la bitácora.
- El modelo aplica restricciones propias (PK, FK, UNIQUE y CHECK sobre estado, precio, stock, cantidad y total) que impiden datos inválidos aunque falle una validación en PL/SQL.
- El caso normal confirma su trabajo con COMMIT solo después de que el procesamiento termina.

Mejora identificada: el procedure no tiene su propia sección EXCEPTION. Si fallara a mitad del recorrido, convendría agregar ROLLBACK en el manejador para deshacer los registros parciales de esa ejecución.

6. Evaluación de objetos PL/SQL

- Explique el rol de procedure, function, package y trigger.
- Fundamente cómo estos objetos permitirían una solución integral.

Respuesta del estudiante:

Los objetos almacenados fueron desarrollados por Pablo Quintino (funciones, procedure y package) y Danyer Córdoba (trigger), e integrados en el script maestro.

Objeto	Nombre	Rol en la solución
FUNCTION	calcular_categoria	Recibe el total comprado y devuelve BRONZE, SILVER, GOLD o PLATINUM. Centraliza la regla de clasificación.
FUNCTION	calcular_beneficio	Recibe el total comprado y devuelve 5, 10, 15 o 20 (%). Se puede usar en bloques, procedures y en SELECT.
PROCEDURE	procesar_beneficios	Recibe sucursal y período; por cada cliente consolida compras, llama a las funciones y graba el resultado en BITACORA_BENEFICIOS.
PACKAGE	techmarket_beneficios	Agrupar las dos funciones y el procedure en una unidad con especificación (interfaz pública) y cuerpo (implementación).
TRIGGER	trg_auditoria_beneficios	AFTER INSERT OR UPDATE FOR EACH ROW sobre la bitácora: registra operación, usuario, fecha y valores anterior (:OLD) y nuevo (:NEW) en AUDITORIA_BENEFICIOS.

Tabla 6. Objetos PL/SQL implementados.

Funciones

```
CREATE OR REPLACE FUNCTION calcular_categoria(  
    p_total_comprado IN NUMBER  
)  
RETURN VARCHAR2  
IS  
BEGIN  
  
    IF p_total_comprado < 300000 THEN  
        RETURN 'BRONZE';  
  
    ELSIF p_total_comprado < 700000 THEN  
        RETURN 'SILVER';  
  
    ELSIF p_total_comprado < 1500000 THEN  
        RETURN 'GOLD';  
  
    ELSE  
        RETURN 'PLATINUM';  
    END IF;  
  
END calcular_categoria;  
  
/
```

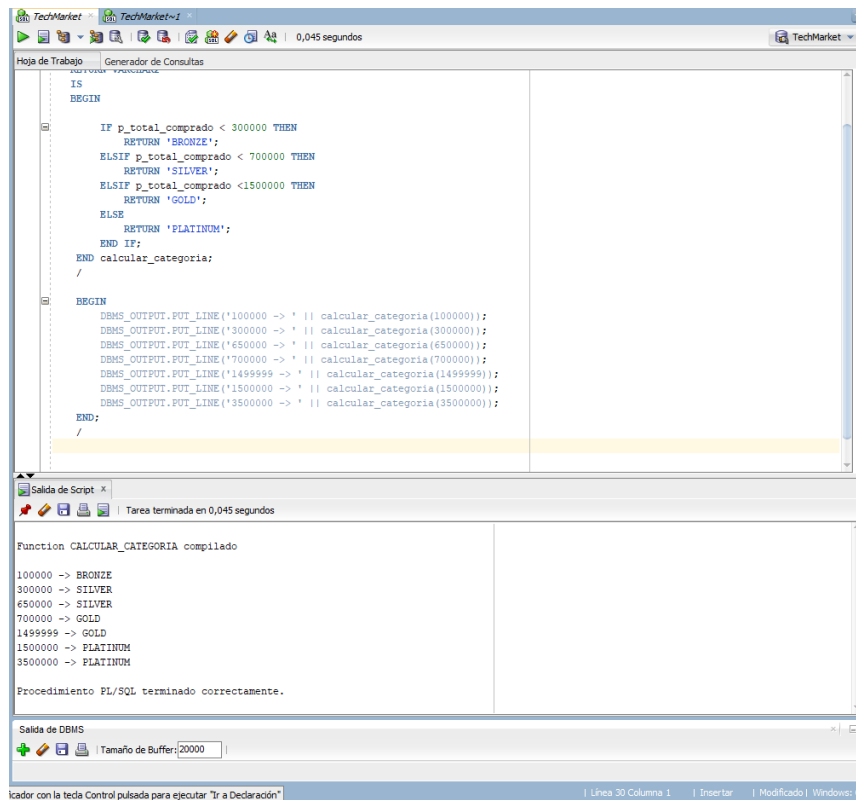


Figura 3. Evidencia: *calcular_categoria* compilada y probada en los límites de cada tramo (SQL Developer).

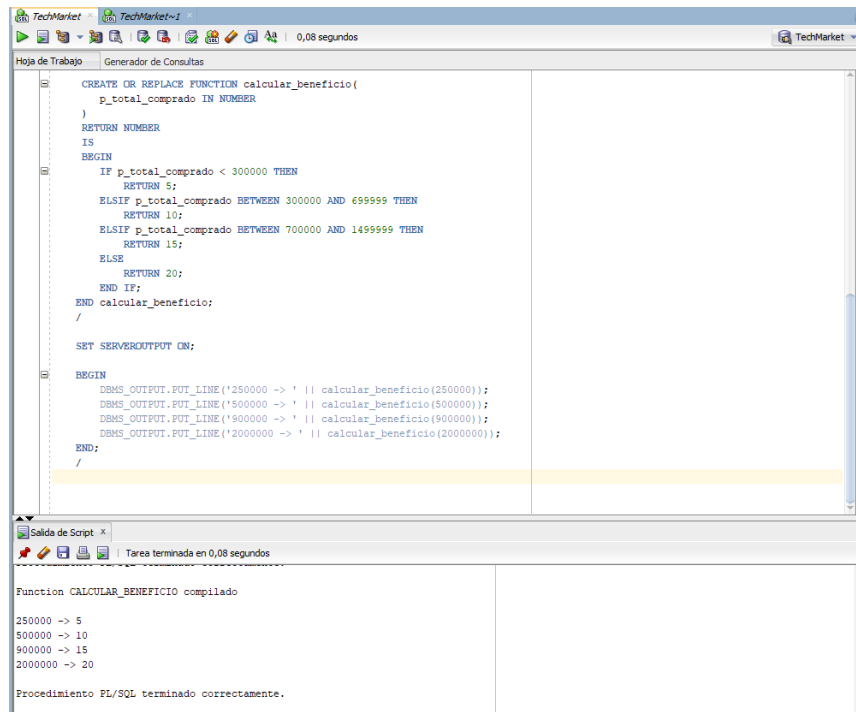


Figura 4. Evidencia: *calcular_beneficio* compilada y probada (SQL Developer).

Nota: la captura de *calcular_beneficio* corresponde a una versión preliminar que usaba **BETWEEN** en los tramos intermedios. La versión final (script maestro) usa comparaciones

con “<”, igual que `calcular_categoria`, y devuelve los mismos resultados para los valores probados (250.000 → 5, 500.000 → 10, 900.000 → 15, 2.000.000 → 20). El cambio corrige un caso borde: con `BETWEEN`, un total con decimales como \$699.999,50 no entraba en ningún tramo y recibía 20%.

Package

```
CREATE OR REPLACE PACKAGE techmarket_beneficios AS

    FUNCTION calcular_categoria(
        p_total_comprado IN NUMBER
    )
    RETURN VARCHAR2;

    FUNCTION calcular_beneficio(
        p_total_comprado IN NUMBER
    )
    RETURN NUMBER;

    PROCEDURE procesar_beneficios(
        p_id_sucursal IN NUMBER,
        p_fecha_desde IN DATE,
        p_fecha_hasta IN DATE
    );

END techmarket_beneficios;

/
```

Trigger

```
CREATE OR REPLACE TRIGGER trg_auditoria_beneficios

AFTER INSERT OR UPDATE

ON bitacora_beneficios

FOR EACH ROW

DECLARE

    v_operacion VARCHAR2(10);

BEGIN

    IF INSERTING THEN

        v_operacion := 'INSERT';

    ELSIF UPDATING THEN

        v_operacion := 'UPDATE';

    END IF;

    INSERT INTO auditoria_beneficios (

        id_bitacora,

        operacion,

        usuario_bd,

        fecha_evento,

        categoria_anterior,

        categoria_nueva,

        total_anterior,

        total_nuevo

    )

    VALUES (
```

ID_BITACORA	ID_CLIENTE	ID_SUCURSAL	PERIODO_DESDE	PERIODO_HASTA	CANTIDAD_COMPRAS	TOTA
USUARIO_BD					FECHA_ACTUALIZACION	
1	101	10	01-AUG-26	31-AUG-26	3	1339950 GOLD
sandbox_user						23-SEP-26
2	102	10	01-AUG-26	31-AUG-26	2	759960 GOLD
sandbox_user						23-SEP-26
3	103	10	01-AUG-26	31-AUG-26	1	219980 BRONZE
sandbox_user						23-SEP-26
4	104	10	01-AUG-26	31-AUG-26	1	1599800 PLATINUM
sandbox_user						23-SEP-26

Evidencia 7. BITACORA_BENEFICIOS después de P1: 4 clientes procesados con su categoría.

¿Cómo forman una solución integral?

Los objetos se complementan en una cadena que sigue la arquitectura del caso (Figura 1):

- **Las funciones** son la única fuente de las reglas de negocio. Si TechMarket cambia un tramo o un porcentaje, se modifica en un solo lugar y todos los procesos que las usan (el bloque RECORD, los cursores y el procedure) quedan actualizados.
- **El procedure** orquesta el proceso: recorre los clientes de una sucursal y período, usa las funciones y persiste el resultado. Queda almacenado en la base de datos y se puede ejecutar cada mes con otros parámetros sin reescribir código.
- **El package** ordena la solución en un módulo (techmarket_beneficios), separa interfaz de implementación, permite controlar permisos sobre una sola unidad y se carga en memoria como un todo. La prueba P1 invoca al procedure a través del package.
- **El trigger** agrega trazabilidad sin depender de quien inserte o modifique la bitácora: aunque alguien actualice un registro manualmente, la auditoría deja constancia del usuario, la fecha y los valores anterior y nuevo.
- **Las excepciones** protegen el proceso completo validando la sucursal, el período y la existencia de ventas antes de procesar.

Límites y mejoras identificadas

- Las funciones y el procedure existen dos veces: como objetos independientes y dentro del package. Convendría dejar solo la versión del package y que los bloques la invoquen como techmarket_beneficios.calcular_categoria, para evitar que las dos copias diverjan.
- Si procesar_beneficios se ejecuta dos veces con los mismos parámetros, duplica los registros en la bitácora. Se podría validar si ya existe un registro para ese cliente, sucursal y período y, en ese caso, actualizarlo (lo que además activaría la rama UPDATE del trigger).
- Las validaciones de P2 a P4 están en bloques separados; integrarlas al inicio del procedure (con RAISE_APPLICATION_ERROR y ROLLBACK) haría que toda ejecución quedara protegida automáticamente.
- La versión del procedure dentro del package no muestra un resumen por cliente con DBMS_OUTPUT (la versión independiente sí lo hace); la evidencia del caso normal se obtiene consultando la bitácora.

7. Resultados y conclusiones

- Resume los resultados obtenidos.
- Indique aprendizajes y mejoras futuras.

Respuesta del estudiante:

Resultados

Para la sucursal Antonio Varas (10) en agosto de 2026, el proceso clasificó a los cuatro clientes con compras según las reglas del negocio:

Cliente	Compras	Total	Categoría	Beneficio
101 Ana Torres	3	\$1.339.950	GOLD	15%
102 Pedro Diaz	2	\$759.960	GOLD	15%
103 Maria Soto	1	\$219.980	BRONZE	5%
104 Carlos Rojas	1	\$1.599.800	PLATINUM	20%

Tabla 7. Resultado del procesamiento (sucursal 10, agosto 2026).

- El RECORD consolidó la ficha comercial del cliente 101 y el VARRAY almacenó el catálogo de cuatro porcentajes de beneficio.
- Los cursores parametrizados con LOOP anidado recorrieron clientes y ventas; al cambiar los parámetros (por ejemplo, sucursal 20) el conjunto de clientes y totales cambia (Tabla 4).
- Las excepciones controlaron una sucursal inexistente (NO_DATA_FOUND), un período inválido y una sucursal sin ventas (excepciones de usuario).
- El package grabó los resultados en BITACORA_BENEFICIOS y el trigger dejó un registro de auditoría por cada inserción.

Aprendizajes

- Llevar las reglas de negocio a funciones almacenadas evita repetir lógica y asegura que todos los procesos clasifiquen igual.
- Los cursores parametrizados permiten reutilizar la misma lógica para cualquier sucursal o período, y el anidamiento modela de forma natural la relación maestro-detalle.
- Diferenciar excepciones predefinidas de Oracle de las definidas por el usuario permite dar mensajes claros tanto para errores técnicos como para reglas del negocio.
- Integrar el trabajo de varios integrantes en un script maestro obligó a unificar versiones: por ejemplo, se consolidó una sola versión de calcular_beneficio y del trigger.

Mejoras futuras

- Usar el VARRAY como fuente del porcentaje de beneficio, en lugar de solo mostrarlo.
- Dejar las funciones y el procedure solo dentro del package, eliminando las copias independientes.
- Agregar a procesar_beneficios validaciones de entrada, manejo de excepciones con ROLLBACK y control de duplicados en la bitácora.
- Considerar el estado del cliente (ACTIVO/INACTIVO) antes de asignar beneficios.

8. Anexos

- Código completo.
- Capturas de pantalla.
- Pruebas realizadas.

Respuesta del estudiante:

Anexo A. Código PL/SQL completo

A continuación se incluye el código PL/SQL del script maestro (secciones 7 a 20). Las secciones 1 a 6 (eliminación y creación de tablas, índices y datos iniciales) corresponden al

script base entregado por el docente, que no fue modificado. Se omitieron solo líneas en blanco consecutivas. Los scripts completos van en la carpeta scripts/ de la entrega.

```
--
=====

-- 7. FUNCTION: CALCULAR_CATEGORIA

--
=====

CREATE OR REPLACE FUNCTION calcular_categoria(
    p_total_comprado IN NUMBER
)
RETURN VARCHAR2
IS
BEGIN
    IF p_total_comprado < 300000 THEN
        RETURN 'BRONZE';

    ELSIF p_total_comprado < 700000 THEN
        RETURN 'SILVER';

    ELSIF p_total_comprado < 1500000 THEN
        RETURN 'GOLD';

    ELSE
        RETURN 'PLATINUM';

    END IF;

END calcular_categoria;

/

--
=====

-- 8. FUNCTION: CALCULAR_BENEFICIO

--
=====

CREATE OR REPLACE FUNCTION calcular_beneficio(
    p_total_comprado IN NUMBER
)
RETURN NUMBER
IS
BEGIN
    IF p_total_comprado < 300000 THEN
        RETURN 5;

    ELSIF p_total_comprado < 700000 THEN
        RETURN 10;

    ELSIF p_total_comprado < 1500000 THEN
        RETURN 15;

    ELSE
        RETURN 20;

    END IF;

END calcular_beneficio;

/

--
=====

-- 9. RECORD + VARRAY

--
=====
```

DECLARE

TYPE cliente_record IS RECORD (

id_cliente CLIENTE.ID_CLIENTE%TYPE,

nombre_cliente CLIENTE.NOMBRE%TYPE,

cantidad_compras NUMBER,

total_comprado NUMBER,

categoria VARCHAR2(20),

beneficio NUMBER

);

v_cliente cliente_record;

TYPE beneficios_varray IS VARRAY(4) OF NUMBER;

v_beneficios beneficios_varray :=

beneficios_varray(5, 10, 15, 20);

BEGIN

SELECT

c.id_cliente,

c.nombre,

COUNT(v.id_venta),

NVL(SUM(v.total), 0)

INTO

v_cliente.id_cliente,

v_cliente.nombre_cliente,

v_cliente.cantidad_compras,

v_cliente.total_comprado

FROM cliente c

LEFT JOIN venta v

ON c.id_cliente = v.id_cliente

WHERE c.id_cliente = 101

GROUP BY

c.id_cliente,

c.nombre;

v_cliente.categoria :=

calcular_categoria(

v_cliente.total_comprado

);

v_cliente.beneficio :=

calcular_beneficio(

v_cliente.total_comprado

);

DBMS_OUTPUT.PUT_LINE(

'ID Cliente: ' ||

v_cliente.id_cliente

);

DBMS_OUTPUT.PUT_LINE(

'Nombre: ' ||

v_cliente.nombre_cliente

);

DBMS_OUTPUT.PUT_LINE(

'Cantidad compras: ' ||

v_cliente.cantidad_compras

);

```

DBMS_OUTPUT.PUT_LINE(
    'Total comprado: $' ||
    v_cliente.total_comprado
);

DBMS_OUTPUT.PUT_LINE(
    'Categoria: ' ||
    v_cliente.categoria
);

DBMS_OUTPUT.PUT_LINE(
    'Beneficio: ' ||
    v_cliente.beneficio || '%'
);

DBMS_OUTPUT.PUT_LINE(
    'Beneficios disponibles: ' ||
    v_beneficios(1) || '%, ' ||
    v_beneficios(2) || '%, ' ||
    v_beneficios(3) || '%, ' ||
    v_beneficios(4) || '%'
);

END;

/

--
=====
-- 10. CURSORES PARAMETRIZADOS + LOOP ANIDADO
--
=====

DECLARE

    v_id_sucursal NUMBER := 10;

    v_fecha_desde DATE :=
        DATE '2026-08-01';

    v_fecha_hasta DATE :=
        DATE '2026-08-31';

    -- CURSOR MAESTRO
    CURSOR cur_clientes(
        p_id_sucursal NUMBER,
        p_fecha_desde DATE,
        p_fecha_hasta DATE
    )
    IS
        SELECT DISTINCT
            c.id_cliente,
            c.nombre
        FROM cliente c

        JOIN venta v
            ON c.id_cliente = v.id_cliente

        WHERE v.id_sucursal = p_id_sucursal

        AND v.fecha_venta
            BETWEEN p_fecha_desde
            AND p_fecha_hasta

        ORDER BY c.id_cliente;

    -- CURSOR DETALLE

```

```

CURSOR cur_ventas(
    p_id_cliente NUMBER,
    p_id_sucursal NUMBER,
    p_fecha_desde DATE,
    p_fecha_hasta DATE
)
IS

    SELECT
        id_venta,
        fecha_venta,
        total

    FROM venta

    WHERE id_cliente = p_id_cliente

    AND id_sucursal = p_id_sucursal

    AND fecha_venta
        BETWEEN p_fecha_desde
        AND p_fecha_hasta

    ORDER BY fecha_venta;

v_cantidad_compras NUMBER;

v_total_comprado NUMBER;

BEGIN

    -- LOOP MAESTRO

    FOR cliente IN cur_clientes(
        v_id_sucursal,
        v_fecha_desde,
        v_fecha_hasta
    )
    LOOP

        v_cantidad_compras :=
            v_cantidad_compras + 1;

        v_total_comprado :=
            v_total_comprado + venta.total;

        DBMS_OUTPUT.PUT_LINE(
            '-----'
        );

        DBMS_OUTPUT.PUT_LINE(
            'Cliente: ' ||
            cliente.id_cliente ||
            ' - ' ||
            cliente.nombre
        );

        -- LOOP DETALLE

        FOR venta IN cur_ventas(
            cliente.id_cliente,
            v_id_sucursal,
            v_fecha_desde,
            v_fecha_hasta
        )
        LOOP

```

```

                                calcular_beneficio(
                                v_total_comprado
                                ) ||
                                '%'
                                );

                                TO_CHAR(
                                venta.fecha_venta,
                                'DD/MM/YYYY'
                                ) ||
                                ' | Total: $' ||
                                venta.total
                                );

END LOOP;

DBMS_OUTPUT.PUT_LINE(
    'Cantidad compras: ' ||
    v_cantidad_compras
);

DBMS_OUTPUT.PUT_LINE(
    'Total comprado: $' ||
    v_total_comprado
);

DBMS_OUTPUT.PUT_LINE(
    'Categoria: ' ||
    calcular_categoria(
        v_total_comprado
    )
);

DBMS_OUTPUT.PUT_LINE(
    'Beneficio: ' ||

```

```

                                END LOOP;

                                END;

                                /

                                --
                                =====
                                =====

                                -- 11. PROCEDURE: PROCESAR_BENEFICIOS

                                --
                                =====
                                =====

CREATE OR REPLACE PROCEDURE procesar_beneficios(
    p_id_sucursal IN NUMBER,
    p_fecha_desde IN DATE,
    p_fecha_hasta IN DATE
)
IS
    v_total_comprado NUMBER;
    v_cantidad_compras NUMBER;
    v_categoria VARCHAR2(20);
    v_beneficio NUMBER;

BEGIN

```

```

FOR cliente IN (

    SELECT

        c.id_cliente,

        c.nombre

    FROM cliente c

    JOIN venta v

        ON c.id_cliente = v.id_cliente

    WHERE v.id_sucursal = p_id_sucursal

    AND v.fecha_venta

        BETWEEN p_fecha_desde

        AND p_fecha_hasta

    GROUP BY

        c.id_cliente,

        c.nombre

    ORDER BY c.id_cliente

)

```

```

LOOP

```

```

SELECT

    COUNT(v.id_venta),

    NVL(SUM(v.total), 0)

INTO

    v_cantidad_compras,

    v_total_comprado

```

```

FROM venta v

WHERE v.id_cliente =

    cliente.id_cliente

AND v.id_sucursal =

    p_id_sucursal

AND v.fecha_venta

    BETWEEN p_fecha_desde

    AND p_fecha_hasta;

v_categoria :=

    calcular_categoria(

        v_total_comprado

    );

v_beneficio :=

    calcular_beneficio(

        v_total_comprado

    );

INSERT INTO bitacora_beneficios (

```

```

    id_cliente,

    id_sucursal,

    periodo_desde,

    periodo_hasta,

    cantidad_compras,

    total_compras,

    categoria_cliente,

    porcentaje_beneficio,

    fecha_proceso,

    usuario_bd,

```

```

        fecha_actualizacion

    )

VALUES (

    cliente.id_cliente,

    p_id_sucursal,

    p_fecha_desde,

    p_fecha_hasta,

    v_cantidad_compras,

    v_total_comprado,

    v_categoria,

    v_beneficio,

    SYSDATE,

    USER,

    SYSDATE

);

DBMS_OUTPUT.PUT_LINE(

    'Cliente: ' ||

    cliente.id_cliente ||

    ' | Compras: ' ||

    v_cantidad_compras ||

    ' | Total: $' ||

    v_total_comprado ||

    ' | Categoria: ' ||

    v_categoria ||

    ' | Beneficio: ' ||

    v_beneficio ||

    '%'

);

END LOOP;

END procesar_beneficios;

/

--
=====

-- 12. PACKAGE

--
=====

CREATE OR REPLACE PACKAGE techmarket_beneficios AS

    FUNCTION calcular_categoria(

        p_total_comprado IN NUMBER

    )

    RETURN VARCHAR2;

    FUNCTION calcular_beneficio(

        p_total_comprado IN NUMBER

    )

    RETURN NUMBER;

    PROCEDURE procesar_beneficios(

        p_id_sucursal IN NUMBER,

        p_fecha_desde IN DATE,

        p_fecha_hasta IN DATE

    );

END techmarket_beneficios;

/

--
=====

-- 13. PACKAGE BODY

```

```

--
=====

=====

CREATE OR REPLACE PACKAGE BODY techmarket_beneficios AS

    FUNCTION calcular_categoria(
        p_total_comprado IN NUMBER
    )
    RETURN VARCHAR2
    IS
    BEGIN

        IF p_total_comprado < 300000 THEN

            RETURN 'BRONZE';

        ELSIF p_total_comprado < 700000 THEN

            RETURN 'SILVER';

        ELSIF p_total_comprado < 1500000 THEN

            RETURN 'GOLD';

        ELSE

            RETURN 'PLATINUM';

        END IF;

    END calcular_categoria;

    FUNCTION calcular_beneficio(
        p_total_comprado IN NUMBER
    )
    RETURN NUMBER
    IS
    BEGIN

        IF p_total_comprado < 300000 THEN

            RETURN 5;

        ELSIF p_total_comprado < 700000 THEN

            RETURN 10;

        ELSIF p_total_comprado < 1500000 THEN

            RETURN 15;

        ELSE

            RETURN 20;

        END IF;

    END calcular_beneficio;

    PROCEDURE procesar_beneficios(
        p_id_sucursal IN NUMBER,
        p_fecha_desde IN DATE,
        p_fecha_hasta IN DATE
    )
    IS
        v_total_comprado NUMBER;

        v_cantidad_compras NUMBER;

        v_categoria VARCHAR2(20);

        v_beneficio NUMBER;

    BEGIN

        FOR cliente IN (

```

```

SELECT
    c.id_cliente,
    c.nombre

FROM cliente c

JOIN venta v
    ON c.id_cliente = v.id_cliente

WHERE v.id_sucursal =
    p_id_sucursal

AND v.fecha_venta
    BETWEEN p_fecha_desde
    AND p_fecha_hasta;

v_categoria :=
    calcular_categoria(
        v_total_comprado
    );

GROUP BY
    c.id_cliente,
    c.nombre

ORDER BY c.id_cliente

)

LOOP

SELECT
    COUNT(v.id_venta),
    NVL(SUM(v.total), 0)

INTO
    v_cantidad_compras,
    v_total_comprado

FROM venta v

WHERE v.id_cliente =
    cliente.id_cliente

AND v.id_sucursal =
    p_id_sucursal

AND v.fecha_venta
    BETWEEN p_fecha_desde
    AND p_fecha_hasta;

v_beneficio :=
    calcular_beneficio(
        v_total_comprado
    );

INSERT INTO bitacora_beneficios (
    id_cliente,
    id_sucursal,
    periodo_desde,
    periodo_hasta,
    cantidad_compras,
    total_compras,
    categoria_cliente,
    porcentaje_beneficio,
    fecha_proceso,
    usuario_bd,
    fecha_actualizacion

```

```

)

VALUES (

    cliente.id_cliente,

    p_id_sucursal,

    p_fecha_desde,

    p_fecha_hasta,

    v_cantidad_compras,

    v_total_comprado,

    v_categoria,

    v_beneficio,

    SYSDATE,

    USER,

    SYSDATE

);

END LOOP;

END procesar_beneficios;

END techmarket_beneficios;

/

--
=====
-- 14. TRIGGER DE AUDITORIA
--
=====

CREATE OR REPLACE TRIGGER trg_auditoria_beneficios

AFTER INSERT OR UPDATE

ON bitacora_beneficios

FOR EACH ROW

DECLARE

    v_operacion VARCHAR2(10);

BEGIN

    IF INSERTING THEN

        v_operacion := 'INSERT';

    ELSIF UPDATING THEN

        v_operacion := 'UPDATE';

    END IF;

    INSERT INTO auditoria_beneficios (

        id_bitacora,

        operacion,

        usuario_bd,

        fecha_evento,

        categoria_anterior,

        categoria_nueva,

        total_anterior,

        total_nuevo

    )

VALUES (

        :NEW.id_bitacora,

        v_operacion,

```

```

        USER,

        SYSDATE,

        :OLD.categoria_cliente,

        :NEW.categoria_cliente,

        :OLD.total_compras,

        :NEW.total_compras

    );

    DBMS_OUTPUT.PUT_LINE(

        'TRAZABILIDAD: Registro automatizado en
AUDITORIA_BENEFICIOS para ID ' ||

        :NEW.id_bitacora

    );

END;

/

--
=====

-- 15. P2 - SUCURSAL INEXISTENTE

-- EXCEPCION PREDEFINIDA NO_DATA_FOUND

--
=====

DECLARE

    v_sucursal_id NUMBER := 999;

    v_sucursal NUMBER;

BEGIN

    SELECT id_sucursal

    INTO v_sucursal

        FROM sucursal

        WHERE id_sucursal = v_sucursal_id;

    DBMS_OUTPUT.PUT_LINE(

        'Sucursal encontrada: ' ||

        v_sucursal

    );

    EXCEPTION

        WHEN NO_DATA_FOUND THEN

            DBMS_OUTPUT.PUT_LINE(

                'ERROR: La sucursal ' ||

                v_sucursal_id ||

                ' no existe en el sistema.'

            );

        WHEN OTHERS THEN

            DBMS_OUTPUT.PUT_LINE(

                'ERROR NO ESPERADO: ' ||

                SQLERRM

            );

END;

/

--
=====

-- 16. P3 - PERIODO INVALIDO

-- EXCEPCION DEFINIDA POR EL USUARIO

```

Subdirección de Diseño Instruccional · Evaluación Parcial N°1 · Kit Estudiante

```

BEGIN
    SELECT COUNT(*)
    INTO v_total_ventas
    FROM venta
    WHERE id_sucursal = v_sucursal_id

    AND fecha_venta
        BETWEEN v_fecha_inicio
        AND v_fecha_fin;

    IF v_total_ventas = 0 THEN

        RAISE ex_sin_ventas;

    END IF;

    DBMS_OUTPUT.PUT_LINE(
        'La sucursal tiene ventas en el periodo.'
    );

EXCEPTION

    WHEN ex_sin_ventas THEN

        DBMS_OUTPUT.PUT_LINE(
            'ERROR: La sucursal ' ||
            v_sucursal_id ||
            ' no registro ventas en el periodo.'
        );

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE(
            'ERROR NO ESPERADO: ' ||
            SQLERRM
        );

    END;

/

--
=====
-- 18. P1 - PROCESAMIENTO NORMAL
--
=====

BEGIN

    techmarket_beneficios.procesar_beneficios(

        10,

        DATE '2026-08-01',

        DATE '2026-08-31'

    );

    END;

/

COMMIT;

```

```
--
=====
=====
```

```
-- 19. CONSULTAR BITACORA
```

```
--
=====
=====
```

```
SELECT
```

```
    id_bitacora,
    id_cliente,
    id_sucursal,
    periodo_desde,
    periodo_hasta,
    cantidad_compras,
    total_compras,
    categoria_cliente,
    porcentaje_beneficio,
    fecha_proceso,
    usuario_bd,
    fecha_actualizacion
```

```
FROM bitacora_beneficios
```

```
ORDER BY id_bitacora;
```

```
--
=====
=====
```

```
-- 20. CONSULTAR AUDITORIA
```

```
--
=====
=====
```

```
SELECT
```

```
    id_auditoria,
    id_bitacora,
    operacion,
    usuario_bd,
    fecha_evento,
    categoria_anterior,
    categoria_nueva,
    total_anterior,
    total_nuevo
```

```
FROM auditoria_beneficios
```

```
ORDER BY id_auditoria;
```

Anexo B. Índice de evidencias

Carpeta	Archivo	Contenido	Sección
record_varray/	01_record_varray.png	Salida del bloque RECORD + VARRAY	3
cursores/	01_cursores_suc10_ago.png	Cursores, sucursal 10, agosto	4
excepciones/	01_P1_caso_normal.png	Procesamiento normal con package	5
excepciones/	02_P2_no_data_found.png	Sucursal inexistente	5
excepciones/	03_P3_periodo_invalido.png	Período inválido	5
excepciones/	04_P4_sin_ventas.png	Sucursal sin ventas	5
objetos_plsql/	01_bitacora.png	Bitácora después de P1	6
objetos_plsql/	02_calcular_categoria.png	Función calcular_categoria (Figura 3)	6
objetos_plsql/	03_calcular_beneficio.png	Función calcular_beneficio (Figura 4)	6

Tabla 8. Evidencias según la estructura de entrega en AVA.

Anexo C. Pruebas realizadas

#	Prueba	Entrada	Resultado esperado
1	calcular_categoria	100.000 / 300.000 / 650.000 / 700.000 / 1.499.999 / 1.500.000 / 3.500.000	BRONZE / SILVER / SILVER / GOLD / GOLD / PLATINUM / PLATINUM
2	calcular_beneficio	250.000 / 500.000 / 900.000 / 2.000.000	5 / 10 / 15 / 20
3	RECORD + VARRAY	Cliente 101	4 compras, \$1.449.940, GOLD, 15%
4	Cursores	Sucursal 10, agosto 2026	4 clientes (Tabla 4)
5	P1 caso normal	Sucursal 10, agosto 2026	4 registros en bitácora + 4 en auditoría
6	P2 NO_DATA_FOUND	Sucursal 999	Mensaje de sucursal inexistente
7	P3 ex_periodo_invalido	31-08-2026 a 01-08-2026	Mensaje de período inválido
8	P4 ex_sin_ventas	Sucursal 20, septiembre 2026	Mensaje de sucursal sin ventas

Tabla 9. Resumen de pruebas.